

Jour 4 : Le Deep Learning, l'évaluation et la mise en production

Re-bonjour à tous !

Hier, on a rempli l'Arène : régression, arbres, Random Forest, boosting, SVM, tout le monde est entré dans le ring et on a sorti un leaderboard. Aujourd'hui, deux gros morceaux.

Le matin, on ajoute un nouveau combattant : le **réseau de neurones**. Attention, on le découvre, on ne le maîtrise pas. L'objectif n'est pas de devenir un expert du Deep Learning en une demi-journée (ça, c'est le programme de 5e année). L'objectif, c'est d'ouvrir la boîte noire : comprendre ce qui se passe à l'intérieur, savoir quand ce combattant gagne, et surtout savoir quand il ne sert à rien (spoiler : sur les petites données tabulaires, un Random Forest le bat de loin et reste explicable).

L'après-midi, on devient des **arbitres sérieux**. Parce qu'avoir cinq algos dans une arène, c'est bien, mais si on les juge mal, on couronne le mauvais champion. On va apprendre à évaluer proprement (train / validation / test, cross-validation, choix de la métrique selon le coût métier), puis à **déployer** le vainqueur pour qu'il vive sur le web ce soir. Et on finit par le QCM.

Pas de panique si "réseau de neurones" semble impressionnant : on construit ça brique par brique.

1 - Révisions rapides

Avant d'attaquer, on remet en place ce qu'on a construit hier, parce qu'on va s'en servir tout l'après-midi.

Sans triche : c'est quoi la différence entre une tâche de classification et une tâche de régression ? Et le Random Forest, il appartient à quelle famille ?

Les points clés de J3 à garder sous la main :

- **Les familles d'algos.** On a vu les régressions (linéaire, polynomiale, logistique), les arbres de décision, le Random Forest (une forêt d'arbres qui votent), le Gradient Boosting (des arbres qui corrigent les erreurs des précédents), les SVM, et le clustering KMeans.
- **Le leaderboard du Fight des IA.** Le rituel de l'Arène : même problème, même découpage des données, on entraîne plusieurs algos et on les classe sur une métrique. Le meilleur n'est pas toujours le plus sophistiqué.
- **Les métriques de jugement, vues hier.** On va les réutiliser, donc rappel express :

Instant rappel : la boîte à métriques de J3

- **Matrice de confusion** : le tableau qui croise prédictions et réalité. Quatre cases : vrais positifs (VP), vrais négatifs (VN), faux positifs (FP, fausse alerte), faux négatifs (FN, raté).
- **Precision** : parmi ce que le modèle a dit "positif", combien l'étaient vraiment. $\frac{VP}{VP + FP}$. Élevée = peu de fausses alertes.
- **Recall (rappel)** : parmi les vrais positifs, combien le modèle en a attrapés. $\frac{VP}{VP + FN}$. Élevé = peu de ratés.

- **F1-score** : la moyenne harmonique de precision et recall, un seul chiffre quand on veut équilibrer les deux.
- **ROC / AUC** : la courbe qui montre le compromis vrais positifs vs faux positifs quand on bouge le seuil de décision. L'AUC (aire sous la courbe) résume ça en un nombre entre 0.5 (au hasard) et 1 (parfait).
- **Courbe de lift** : de combien le modèle fait mieux que le hasard quand on cible les cas qu'il juge les plus probables. Très utilisée en marketing et en scoring.

Si un de ces termes est flou, c'est le moment de couper la parole : on va les manipuler concrètement cet après-midi quand on choisira la bonne métrique pour un vrai problème.

Planning de la Journée

1. **L'accroche** : d'où vient le Deep Learning et pourquoi il explose maintenant
 2. **L'architecture d'un réseau** : le perceptron multicouche (PMC), entrée, couches cachées, sortie
 3. **Les fonctions d'activation** : sigmoïde, ReLU, tanh, softmax, et laquelle choisir où
 4. **Comment un réseau apprend de ses erreurs** : fonction de coût, backpropagation, descente de gradient (en intuition)
 5. **Le PMC en vrai** : le construire en Keras, le lancer, comparer au ML classique
 6. **L'après-midi (À votre tour)** : évaluer rigoureusement, déployer le champion, et le QCM
-

2 - L'accroche : d'où vient le Deep Learning

Question rapide pour démarrer : à votre avis, l'idée du neurone artificiel, ça date de quand ? Cette année ? Il y a 10 ans ?

La réponse surprend toujours : 1943. L'idée a presque 80 ans. Alors pourquoi tout le monde n'en parle que depuis une dizaine d'années ? C'est la vraie question, et la réponse explique tout le domaine.

L'essentiel

Le Deep Learning, ce sont des **réseaux de neurones à plusieurs couches**. L'idée de base est vieille, mais elle dormait. Voici les jalons à connaître :

- **1943** : McCulloch et Pitts modélisent un premier neurone artificiel sur le papier. Pure théorie.
- **1958** : le [perceptron](#) de Rosenblatt, le premier neurone qui apprend vraiment. Énorme hype... puis déception : un seul neurone ne sait même pas apprendre la fonction XOR (le "ou exclusif"). Premier hiver de l'IA.
- **1986** : [Rumelhart, Hinton et Williams](#) popularisent la **backpropagation**, l'algorithme qui permet d'entraîner des réseaux à plusieurs couches. La porte s'ouvre, mais les ordinateurs sont trop lents.
- **2012** : un réseau profond ([AlexNet](#)) écrase la concurrence sur un grand concours de reconnaissance d'images. C'est le big bang du Deep Learning moderne. Tout le monde s'y met.

C'est quoi le perceptron ? Le neurone artificiel de base : il prend des entrées, les pondère, les additionne, et décide d'une sortie. C'est la brique élémentaire. Empilez-en plusieurs sur plusieurs couches et vous obtenez un perceptron multicouche (PMC), le réseau qu'on découvre aujourd'hui.

Alors pourquoi 2012 et pas 1986 ? Trois ingrédients se sont alignés, et ce sont eux les vraies raisons du succès actuel :

- **Les données massives.** Un réseau profond a besoin d'énormément d'exemples pour apprendre. Internet, les smartphones, les capteurs ont créé des montagnes de données étiquetées (pensez aux milliards de photos taggées sur le web). Avant, on n'avait pas de quoi nourrir la bête.
- **La puissance de calcul (GPU).** Entraîner un réseau, c'est des milliards de multiplications de matrices. Les **GPU** (cartes graphiques, conçues à l'origine pour les jeux vidéo) font exactement ce type de calcul en parallèle, des centaines de fois plus vite qu'un processeur classique. Sans GPU, pas de Deep Learning à l'échelle.
- **Les frameworks.** Avant, coder un réseau, c'était des centaines de lignes de calcul de dérivées à la main. Aujourd'hui, des outils comme [TensorFlow](#) / [Keras](#) (Google) ou [PyTorch](#) (Meta) font tout le sale boulot. On décrit le réseau en quelques lignes, le framework calcule les dérivées et gère le GPU.

GPU vs CPU, en une phrase ? Le CPU (processeur classique) est un excellent généraliste qui fait peu de choses à la fois mais très bien. Le GPU est une armée de petits calculateurs qui font tous la même opération en même temps : parfait pour les multiplications de matrices d'un réseau de neurones.

Donnée + calcul + outils. Retenez ces trois mots : c'est la réponse exacte à "pourquoi le DL maintenant ?", et elle tombe souvent en entretien.

Situons les jalons. Trois événements dans le désordre, essayons de les remettre dans l'ordre de tête avant de lire la suite :

- la backpropagation est popularisée
- AlexNet gagne le concours ImageNet
- le premier perceptron de Rosenblatt

Dans l'ordre : le perceptron de Rosenblatt (1958) d'abord, la backpropagation (1986) ensuite, AlexNet (2012) en dernier. Et le plus intéressant, c'est pourquoi aucun n'a suffi seul. Le perceptron ne savait même pas faire un XOR. La backpropagation débloquent les couches multiples, mais les machines de 1986 étaient trop lentes et on manquait de données. Il a fallu attendre 2012 pour que les trois ingrédients (données, GPU, frameworks) s'alignent enfin. Chaque jalon posait une brique, aucun ne tenait debout tout seul.

Pour les curieux

Un détail qui éclaire toute l'histoire. Pendant des décennies, les réseaux de neurones étaient mal vus dans la recherche : on les jugeait peu sérieux face aux méthodes statistiques "propres".

Quelques chercheurs ont quand même continué dans le désert : Hinton, LeCun, Bengio. Ils ont touché le prix Turing en 2018 (l'équivalent du Nobel en informatique) précisément pour cette obstination.

Quand les trois ingrédients se sont alignés en 2012, ces gens-là avaient vingt ans d'avance sur tout le monde.

La leçon pro, et elle dépasse le Deep Learning : ce n'est pas parce qu'une idée semble enterrée qu'elle est morte. Elle attend parfois juste que le contexte technique la rattrape.

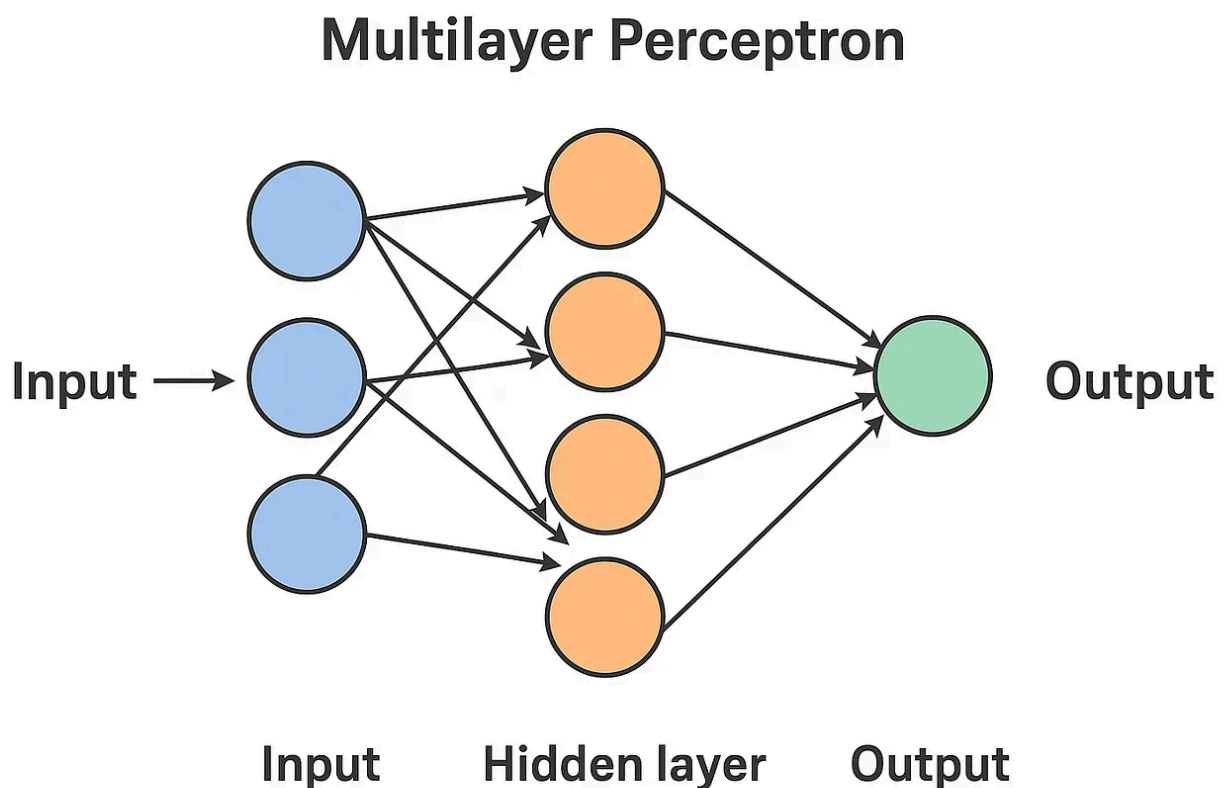
Et un garde-fou tout de suite, parce qu'on va le marteler toute la journée : le Deep Learning n'est PAS toujours la meilleure réponse. Sur une image, du son, du texte, oui, il domine. Sur un tableau de chiffres (un export Excel, une base clients), un Random Forest est souvent meilleur, plus rapide à entraîner, et capable d'expliquer ses décisions. Ce cours met le ML classique au centre pour cette raison exacte.

On sait d'où ça vient et pourquoi ça marche maintenant. La question logique : c'est quoi, concrètement, l'intérieur d'un réseau ? On ouvre le capot.

3 - L'architecture d'un réseau : le perceptron multicouche (PMC)

On a un tableau de données : la surface d'un appartement, son nombre de pièces, son quartier. On veut prédire le prix. Comment une machine transforme ces trois chiffres en une estimation ? Qu'est-ce qui se passe entre l'entrée et la sortie ?

C'est exactement ce qu'un PMC fait. Avant de coder, comprenons ce qu'il y a dans la boîte.



L'essentiel

Un **PMC** (Perceptron Multi-Couches, ou MLP en anglais pour Multi-Layer Perceptron) est un réseau de neurones où l'information circule dans une seule direction : de l'entrée vers la sortie, sans boucle.

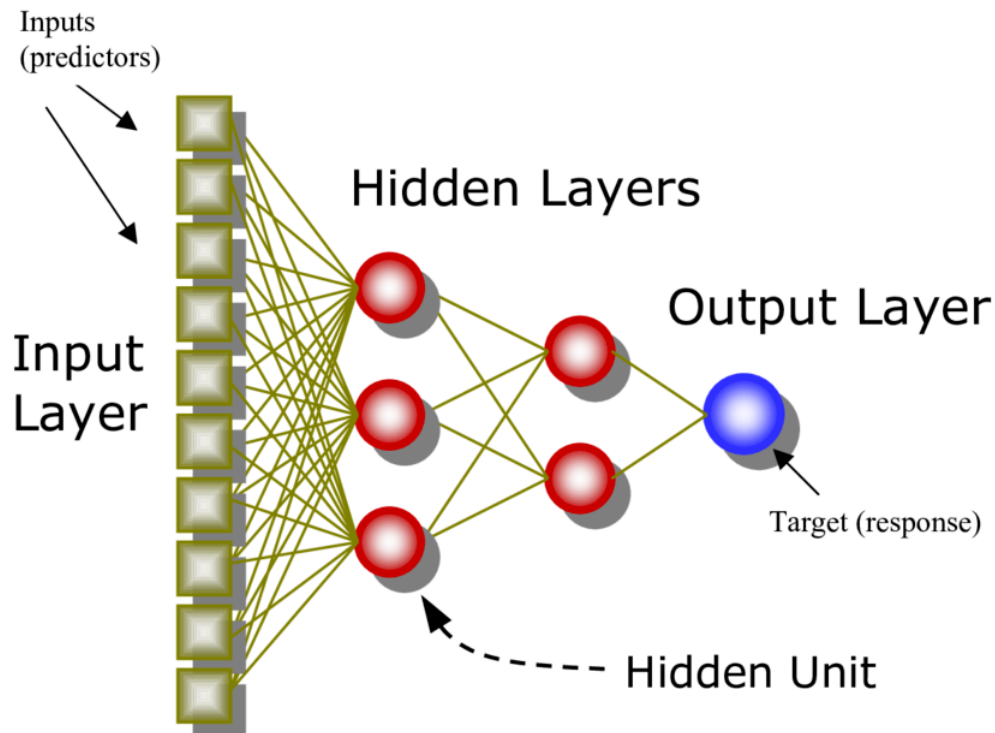
Réseau feed-forward ? "Feed-forward" veut dire que l'info avance tout droit : entrée puis couches cachées puis sortie, jamais en arrière. C'est le type de réseau le plus simple, et le seul qu'on voit aujourd'hui. (Les réseaux qui rebouclent sur eux-mêmes, pour le texte ou les séries temporelles, c'est un autre sujet, pour plus tard.)

Le neurone, la brique de base. Un neurone reçoit plusieurs chiffres en entrée, les combine, et produit un chiffre en sortie. La combinaison, c'est une somme pondérée plus un décalage :

```
z = x1*w1 + x2*w2 + b
sortie = f(z)
```

- `x1, x2` : les entrées (les données)
- `w1, w2` : les **poids** (weight), l'importance que le neurone donne à chaque entrée
- `b` : le **biais**, un décalage qui permet au neurone de s'activer même quand les entrées sont nulles
- `f` : la **fonction d'activation** (on la détaille en section 4)

L'analogie : le poids, c'est le volume d'un canal. Poids à 0.9 = "ce signal compte beaucoup". Poids à 0 = "ce signal n'existe pas pour moi". Le biais, c'est le réglage de base du neurone.



Voyons un seul neurone tourner en [numpy](#). Le code va faire trois choses : poser deux entrées, leur coller des poids et un biais (fixés à la main pour la démo, alors que dans un vrai réseau ils seraient appris), puis calculer la somme pondérée `z` :

```
import numpy as np

# Les données d'entrée : 2 caractéristiques (features)
x = np.array([0.5, -0.3])

# Les poids et le biais (ici fixés à la main pour la démo,
# dans un vrai réseau ils sont appris)
w = np.array([0.8, 0.4])
b = 0.1

# La somme pondérée : on multiplie chaque entrée par son poids et on additionne
# np.dot fait (0.5*0.8) + (-0.3*0.4) en une opération
z = np.dot(x, w) + b

print(f"z = {z:.4f}")
```

Devrait afficher :

```
z = 0.2800
```

C'est quoi `np.dot` ? Le produit scalaire : on multiplie deux listes terme à terme et on additionne tout.
Comme un ticket de caisse : quantités `[5, 3]` fois prix `[0.8, 0.4]` donne `(5*0.8) + (3*0.4) = 5.2`.
Dans le neurone, c'est entrées fois poids.

Mais `x1 x2` ça correspond à quoi en fait ?

Pour réussir à se projeter, ce qu'on a fait en haut, Admettons qu'on veut prédire si un client va résilier.

- `x1 = 0.5` : son ancienneté (normalisée)
- `x2 = -0.3` : son montant mensuel (normalisé)

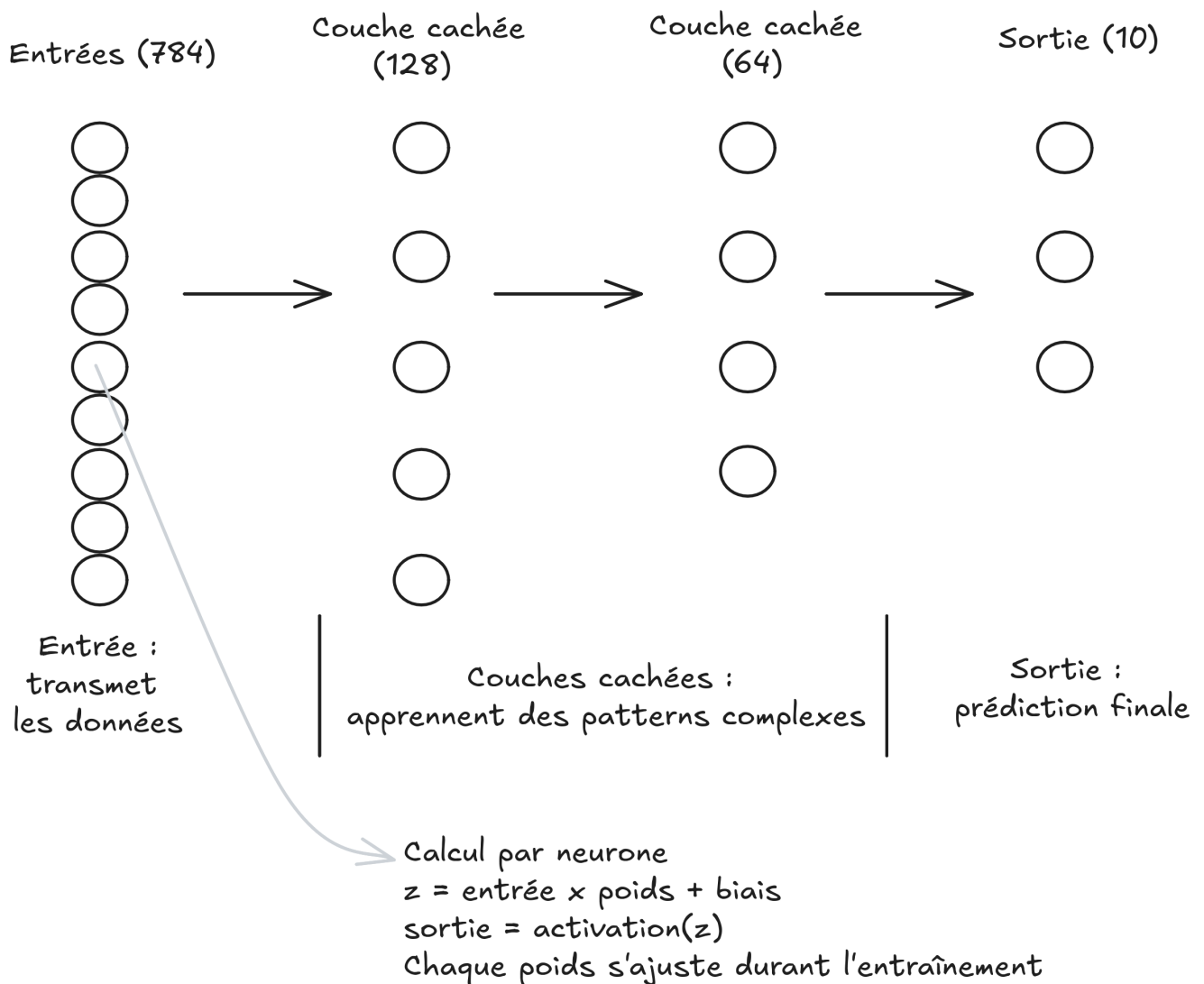
Deux chiffres = deux colonnes du Telco pour un seul client. C'est exactement ça.

D'une brique à un réseau. Empilez des neurones en couches et vous avez un PMC :

- **Couche d'entrée** : reçoit les données brutes. Elle ne transforme rien, elle transmet. Autant de neurones que de features.
- **Couche(s) cachée(s)** : le coeur. C'est là que le réseau apprend des représentations de plus en plus abstraites. Plus de couches et de neurones = plus de capacité à capter des motifs complexes.
- **Couche de sortie** : produit la prédiction. Sa taille et son activation dépendent de la tâche.

Une feature ? Une caractéristique, une variable de vos données. Une image 28x28 pixels = 784 features (un pixel = une feature), donc 784 neurones en entrée. Un appartement décrit par surface, pièces, quartier, âge = 4 features, donc 4 neurones en entrée.

Réseau de neurones : flux de données



Le forward pass. Faire passer les données à travers le réseau, couche par couche, jusqu'à la prédiction, ça s'appelle le forward pass (la passe avant). À chaque couche : `sortie = activation(données * poids + biais)`, puis cette sortie devient l'entrée de la couche suivante. Une chaîne de montage : la donnée entre, chaque poste la transforme, la prédiction sort au bout.

Le preprocessing selon la tâche. Voici le point qui revient en entretien : on configure la couche de sortie selon ce qu'on veut prédire.

Tâche	Sortie attendue	Nb neurones de sortie	Activation de sortie
Régression	Un nombre (ex : un prix)	1	Aucune (linéaire)
Classification binaire	Probabilité d'être dans la classe 1	1	Sigmoïde
Classification multiclass	Probabilité pour chaque classe	N (une par classe)	Softmax

Et côté entrée, une règle d'or vue toute la semaine : on **normalise** les données (les ramener sur une échelle commune, par exemple entre 0 et 1) avant de les envoyer au réseau. Un réseau déteste recevoir une feature qui va de 0 à 1 à côté d'une autre qui va de 0 à 1 000 000 : il n'arrive pas à apprendre. Pour le multiclass, on encode aussi la cible en **one-hot** : la classe "2" parmi 4 devient `[0, 0, 1, 0]`.

One-hot, c'est quoi ? Bon on l'a vu hier mais je le remets. Transformer une étiquette en vecteur de 0 avec un seul 1 à la position de la classe. "chien" parmi (chat, chien, oiseau) devient `[0, 1, 0]`. Ça permet au réseau de comparer sa sortie (des probabilités) à la vérité (un 1 et des 0).

Comptons les paramètres. Prenons un tout petit réseau : 4 features d'entrée vers 5 neurones cachés vers 1 sortie. Essayons de deviner combien il a de paramètres à apprendre avant de lire la réponse (indice : chaque entrée est reliée à chaque neurone caché).

On va remarquer que ça grimpe vite. Entre l'entrée et la couche cachée : 4 entrées x 5 neurones = 20 poids, plus 5 biais (un par neurone caché) = 25. Entre la couche cachée et la sortie : 5 x 1 = 5 poids, plus 1 biais = 6. Total : 31 paramètres pour un réseau ridiculement petit. Si on gonflait les couches on passerait vite à des millions. C'est exactement pour ça qu'un réseau a besoin de beaucoup de données : il a énormément de boutons à régler, et chaque bouton demande des exemples pour trouver sa bonne valeur.

Pour aller plus loin

"Mais concrètement, c'est quoi un poids appris ?"

Reprenons l'appartement. Un neurone caché regarde surface et nombre de pièces. S'il apprend `w_surface = 0.95` et `w_pieces = 0.02`, il a décidé que la surface est presque le seul signal qui l'intéresse. Un autre neurone de la même couche apprendra peut-être `w_surface = 0.1` et `w_pieces = 0.8` : lui se concentre sur les pièces. C'est la diversité de ces "regards" qui permet au réseau de capturer la complexité d'un marché. Personne ne programme ces poids : ils émergent de l'entraînement, qu'on voit en section 5.

Un résultat théorique rassurant, le [théorème d'approximation universelle](#) (Cybenko, 1989) : un PMC avec une seule couche cachée et assez de neurones peut approximer n'importe quelle fonction continue. Traduction : si un motif existe dans vos données, un réseau assez grand peut le trouver. En pratique, "assez grand" peut vouloir dire énorme, et on préfère souvent empiler plusieurs couches plutôt qu'en élargir une seule. Mais ça répond à "pourquoi ça marche ?".

À mater ce soir : [3Blue1Brown - Neural Networks](#) : la meilleure intuition visuelle du forward/backprop, animée. [StatQuest - Neural Networks Pt.1](#) (~20 min) : même sujet, approche plus statistique, idéal pour consolider ce qu'on vient de voir.

On sait comment le signal traverse le réseau. Mais j'ai glissé un ϵ mystérieux dans chaque neurone : la fonction d'activation. Sans elle, tout ce bel empilement de couches s'écroule sur lui-même. Voyons pourquoi.

4 - Les fonctions d'activation

Voilà le problème, et il est élégant : **empiler deux couches linéaires, ça reste une seule couche linéaire.**

L'essentiel des fonctions d'activation

Pourquoi ? Si chaque couche ne fait que `poids * entrée + biais`, alors :

```
Couche 1 : z1 = W1 * x + b1
Couche 2 : z2 = W2 * z1 + b2

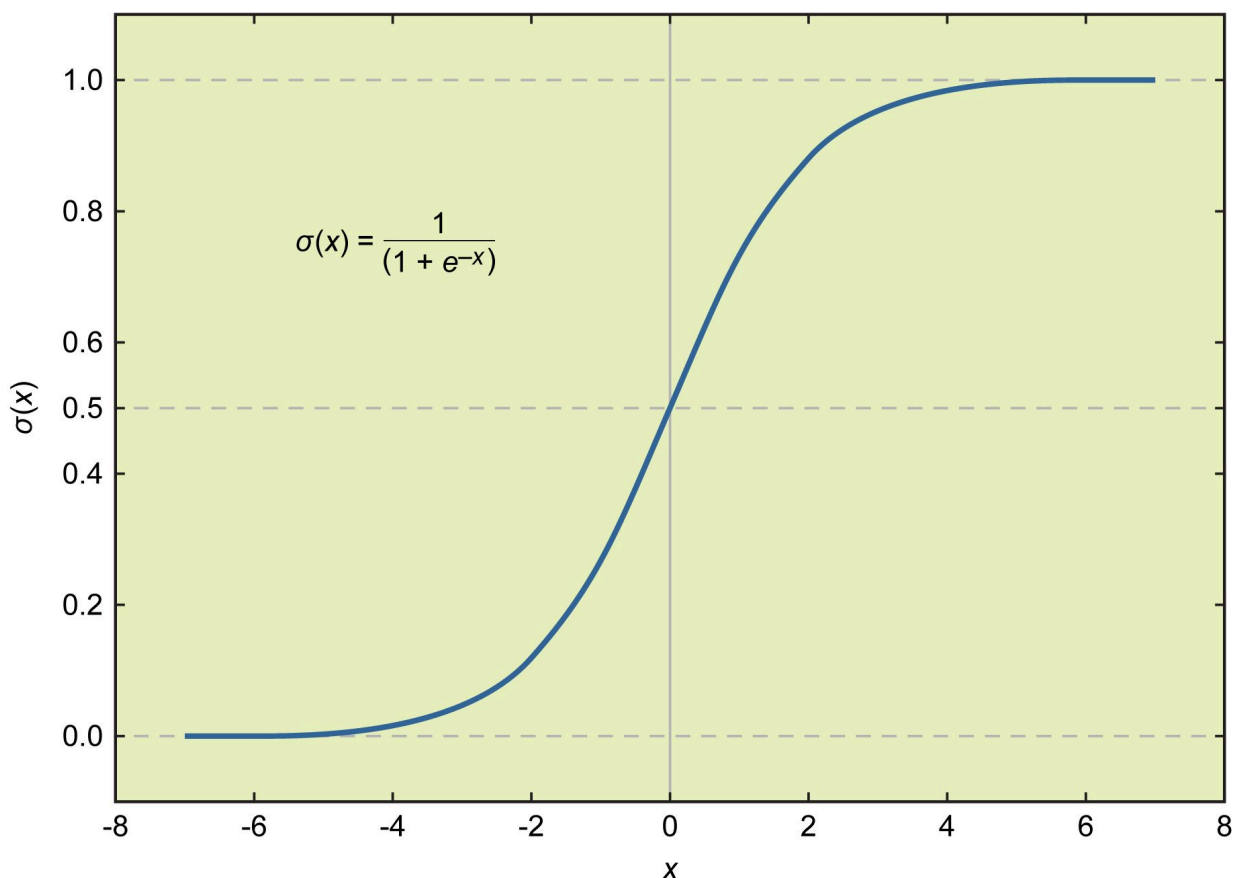
En développant :
z2 = W2 * (W1 * x + b1) + b2
    = (W2 * W1) * x + (W2 * b1 + b2)
    = W' * x + b'
```

C'est juste une multiplication et une addition. Pareil qu'une seule couche. On aurait beau empiler 100 couches, le réseau ne saurait séparer que des données alignées en ligne droite. Or les vrais problèmes (reconnaître une image, le fameux XOR) sont **non-linéaires** : ils demandent des courbes, des zigzags.

La solution : glisser une fonction non-linéaire, l'**activation**, entre les couches. Elle casse la linéarité et débloque tout. Les quatre à connaître :

Sigmoïde : `sigmoid(z) = 1 / (1 + e-z)`. Écrase n'importe quel nombre entre 0 et 1. On la lit comme une probabilité. Usage : couche de sortie en classification binaire.

Sigmoid function



```
import numpy as np

def sigmoid(z):
    # Ramène toute valeur dans ]0, 1[, interprétable comme une probabilité
    return 1 / (1 + np.exp(-z))

print(sigmoid(0))      # 0.5   (au milieu)
print(sigmoid(10))     # ~0.99995 (quasi certain)
print(sigmoid(-10))    # ~0.00005 (quasi exclu)
```

Devrait afficher :

```
0.5
0.9999546021312976
4.5397868702434395e-05
```

ReLU (Rectified Linear Unit) : $\text{relu}(z) = \max(0, z)$. La plus simple qui existe : positif, ça passe tel quel ; négatif, ça devient 0. C'est l'activation par défaut des couches cachées, parce qu'elle entraîne les réseaux beaucoup plus vite que la sigmoïde.

```
def relu(z):
    # Laisse passer le positif, écrase le négatif à zéro
    return np.maximum(0, z)

print(relu(-2))    # 0
print(relu(0.5))   # 0.5
print(relu(3))     # 3
```

tanh (tangente hyperbolique) : variante de la sigmoïde centrée sur 0, sortie entre -1 et 1. Centrée sur 0 = gradients plus stables. Un choix correct en couche cachée si on ne veut pas de ReLU.

```
def tanh(z):
    return np.tanh(z)    # numpy l'a en natif
```

Softmax : $\text{softmax}(z_i) = e^{z_i} / \text{somme}(e^{z_j})$. Transforme un vecteur de scores en distribution de probabilités : chaque valeur entre 0 et 1, et la somme du vecteur fait exactement 1. Usage exclusif : couche de sortie en classification multiclass.

```
def softmax(z):
    # On soustrait le max avant l'exponentielle : stabilité numérique,
    # sans changer le résultat (la soustraction s'annule dans la division)
    exp_z = np.exp(z - np.max(z))
    return exp_z / exp_z.sum()

print(softmax(np.array([2.0, 1.0, 0.1])))
```

Devrait afficher (les trois valeurs somment à 1) :

```
[0.65900114 0.24243297 0.09856589]
```

Pourquoi softmax et pas trois sigmoïdes pour du multiclass ? Parce que la sigmoïde appliquée séparément à chaque classe peut donner une somme de probabilités qui ne fait pas 1. Softmax force une **compétition** : si le réseau est 99% sûr que c'est un "3", il ne peut pas être en même temps 80% sûr que c'est un "5". La somme = 1 oblige les classes à se partager le gâteau.

La table de décision à mémoriser :

Position	Activation
Couches cachées	ReLU (par défaut)
Sortie, régression	Aucune (linéaire)
Sortie, classification binaire	Sigmoïde
Sortie, classification multiclass	Softmax

On bouge un paramètre : reprenons la fonction `sigmoid` et passons-lui un grand vecteur, `sigmoid(np.array([-20, -5, 0, 5, 20]))`, puis regardons les valeurs aux extrêmes. On va remarquer un truc important sur `-20` et `20` : elles sont collées à 0 et à 1, presque plates, alors qu'autour de 0 la courbe bouge franchement. Gardons cette observation en tête, elle explique le point suivant.

On va plus loin

Cette platitude aux extrêmes de la sigmoïde a un nom et une conséquence : le **vanishing gradient** (gradient qui disparaît). Quand la courbe est plate, sa pente (le gradient) est quasi nulle. Et comme l'apprentissage remonte ces pentes en les multipliant couche par couche, des pentes minuscules se multiplient et tendent vers zéro : les premières couches n'apprennent plus rien. C'est précisément ce problème qui a bloqué les réseaux profonds pendant des années.

La solution est arrivée en 2011 : [Glorot, Bordes et Bengio](#) ont montré que ReLU, avec sa pente constante de 1 dans le positif, évitait ce piège et permettait d'entraîner des réseaux profonds. Trois caractères, `max(0, z)`, qui ont débloqué le champ. La leçon : parfois la solution la plus bête bat des années de formules savantes.

ReLU a quand même son défaut, le **dying ReLU** : un neurone dont les entrées deviennent toujours négatives sort toujours 0, son gradient devient 0, et il ne se met plus jamais à jour. Il est "mort". Des variantes existent (Leaky ReLU laisse passer un mince filet pour le négatif), mais c'est de la culture : pour aujourd'hui, ReLU en couches cachées et on avance.

On a un réseau qui calcule une prédiction, avec des activations qui le rendent puissant. Mais au démarrage, ses poids sont aléatoires : il dit n'importe quoi. La vraie magie, c'est comment il corrige ses erreurs tout seul. C'est le coeur de l'affaire.

5 - Comment un réseau apprend de ses erreurs

Scénario : votre réseau prédit 0.9, très confiant, "c'est un chat". La vraie réponse : c'est un chien (étiquette 0). Est-ce que l'erreur doit coûter cher, ou pas grave ?

Cher. Très cher. Une erreur faite avec assurance doit être punie plus fort qu'une hésitation. C'est exactement ce que mesure la **fonction de coût**, le point de départ de l'apprentissage.

L'essentiel

Étape 1 : mesurer l'erreur avec une fonction de coût (la loss). La loss, c'est un chiffre : plus il est grand, plus le réseau se trompe. On en choisit une selon la tâche.

- **RMSE** (Root Mean Square Error), pour la **régression**. L'écart moyen entre prédictions et vraies valeurs, le carré pénalisant fort les grosses erreurs. `RMSE = racine( moyenne( (vrai - prédit)^2 ) )`. Intuition : prédire 250 000 pour un bien à 300 000 donne une erreur de 50 000, et le carré fait mal.
- **Binary Cross-Entropy** (BCE), pour la **classification binaire**. Elle mesure la "surprise" du modèle face à la bonne réponse. Si le réseau prédit 0.9 alors que c'est bien la classe 1 : $-\log(0.9) \approx 0.1$, faible surprise, faible loss. S'il prédit 0.1 pour la classe 1 : $-\log(0.1) \approx 2.3$, grosse surprise, loss élevée. Une erreur confiante explose. C'est exactement le scénario du chat/chien d'au-dessus.
- **Categorical Cross-Entropy** (CCE), pour la **classification multiclass**. Même principe, mais pour N classes. CCE va toujours de pair avec softmax : softmax sort une distribution de probabilités, CCE mesure à quel point elle s'éloigne de la vraie classe (encodée en one-hot).

La loss, en une phrase : un thermomètre de l'erreur. Tout l'entraînement consiste à le faire descendre.

Étape 2 : trouver dans quelle direction corriger, avec le gradient.

Le **gradient** d'un poids, c'est la réponse à : "si je touche un peu ce poids, la loss monte ou descend, et de combien ?" S'il faut diminuer la loss et que la touche le fait monter, c'est qu'il faut aller dans l'autre sens. Le gradient donne la direction et l'ampleur de la correction, pour chaque poids.

Le gradient, sans les maths : imaginez que vous êtes sur une colline dans le brouillard et que vous voulez descendre. Vous tâtez le sol autour de vos pieds pour sentir où ça descend le plus, et vous faites un pas dans cette direction. Le gradient, c'est cette sensation de pente. La loss, c'est l'altitude. On veut atteindre le fond de la vallée.

Étape 3 : propager la correction à tous les poids, avec la backpropagation.

Le problème : un poids tout au début du réseau influence la loss indirectement, à travers toutes les couches qui le suivent. Comment savoir de combien le corriger ?

Backpropagation ? L'algorithme qui calcule, en remontant de la sortie vers l'entrée, la part de responsabilité de chaque poids dans l'erreur finale. Il distribue la "facture" de l'erreur à chacun. Techniquement c'est de la dérivation en chaîne, mais l'intuition suffit aujourd'hui : on rétro-propage le blâme.

Étape 4 : appliquer la correction, avec l'optimiseur.

L'optimiseur fait le pas : il modifie chaque poids dans la direction donnée par son gradient, d'une certaine longueur de pas appelée le **learning rate**.

```
nouveau_poids = ancien_poids - learning_rate * gradient
```

Petit learning rate = petits pas prudents, lent mais sûr. Grand learning rate = grands pas, rapide mais risque de rater le fond de la vallée et de sautiller. Les deux optimiseurs à connaître :

- **SGD** (Stochastic Gradient Descent, descente de gradient stochastique) : la version de base. On prend un petit paquet de données (un mini-batch), on calcule le gradient dessus, on fait un pas. "Stochastic" parce qu'on ne regarde pas tout le dataset d'un coup, mais des paquets aléatoires. Ça ajoute du bruit, qui aide souvent à mieux généraliser.
- **Adam** (Adaptive Moment Estimation, [Kingma et Ba, 2014](#)) : SGD avec de la mémoire. Il se souvient de la direction des gradients récents et adapte la taille du pas tout seul. Résultat : il converge plus vite dans la plupart des cas. C'est le défaut raisonnable pour commencer.

La boucle d'entraînement complète, qu'on répète des dizaines de fois (chaque tour complet sur les données = une **epoch**) :

```
1. Forward pass      : les données traversent le réseau, on obtient une prédiction
2. Calcul de la loss : on compare prédiction et vraie réponse
3. Backpropagation   : on remonte le gradient de la loss vers chaque poids
4. Mise à jour       : l'optimiseur corrige les poids
... on recommence, et la loss descend, epoch après epoch
```

Voyons cette boucle en numpy, sur le neurone le plus simple possible (une seule entrée, un seul poids), pour voir la loss descendre sous nos yeux. Le code va enchaîner exactement les quatre étapes du dessin :

- poser un jeu jouet où la vraie relation est $y = 2 \times x$ (donc on sait que le bon poids vaut 2),
- partir d'un poids faux exprès ($w = 0$) pour avoir quelque chose à corriger,
- à chaque epoch : faire le forward pass ($w \times x$), mesurer la loss, calculer le gradient, puis corriger w d'un pas,
- afficher w et la loss à chaque tour pour voir l'un monter vers 2 et l'autre s'effondrer.

```
import numpy as np

# Un jeu jouet : on veut que le réseau apprenne y = 2*x
X = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([2.0, 4.0, 6.0, 8.0])

w = 0.0          # poids initial (faux exprès)
learning_rate = 0.01

for epoch in range(5):
    y_pred = w * X          # forward pass
    loss = np.mean((y_pred - y) ** 2)  # loss (erreur quadratique moyenne)
    grad = np.mean(2 * (y_pred - y) * X)  # gradient de la loss par rapport à w
    w = w - learning_rate * grad          # mise à jour : on descend la pente

    print(f"epoch {epoch} | w = {w:.3f} | loss = {loss:.3f}")
```

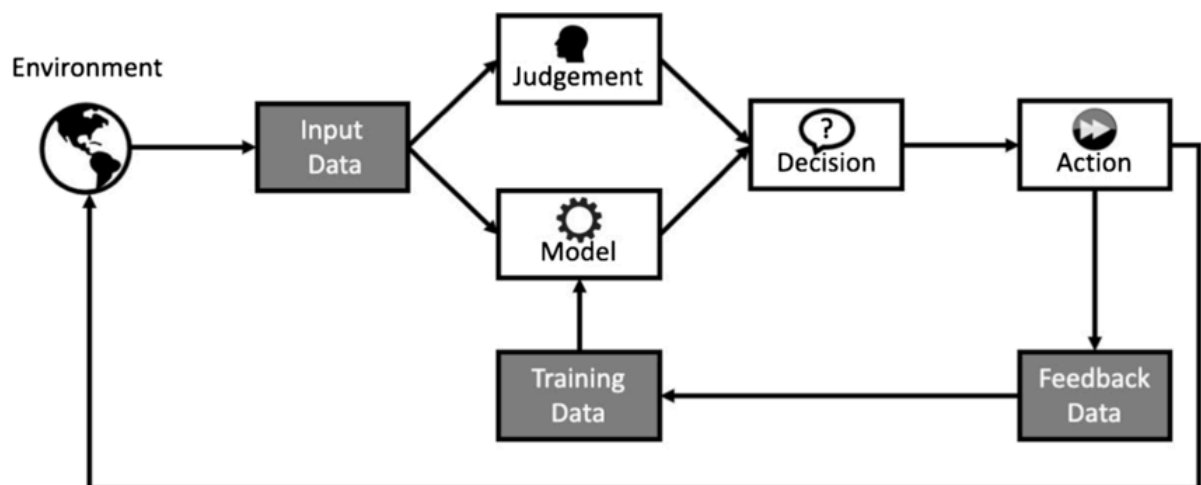
Devrait afficher une loss qui chute et un w qui s'approche de 2 :

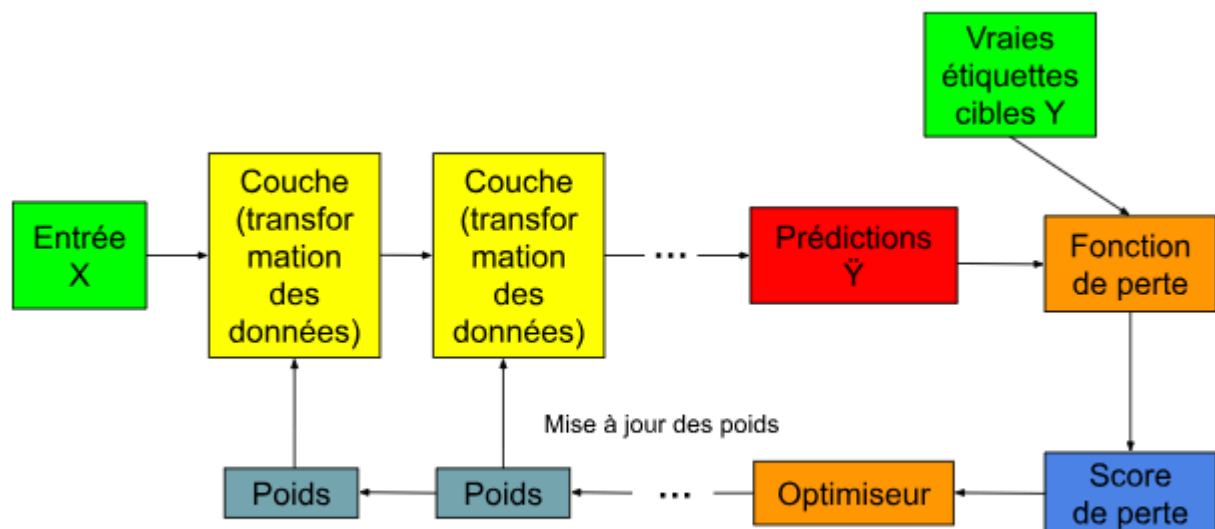
```
epoch 0 | w = 0.300 | loss = 30.000
epoch 1 | w = 0.531 | loss = 17.787
epoch 2 | w = 0.709 | loss = 10.546
epoch 3 | w = 0.846 | loss = 6.253
epoch 4 | w = 0.951 | loss = 3.708
```

La loss descend, w monte vers 2. C'est ça, "apprendre" : répéter la boucle jusqu'à ce que l'erreur soit petite. Tout le Deep Learning tient dans ces quatre étapes.

Manip express : dans le code, faisons passer `learning_rate` de 0.01 à 0.1 et relançons. On va remarquer que la loss plonge beaucoup plus vite, w atteint 2 en quelques epochs au lieu de traîner. Tentant de monter encore. Sauf que si on pousse à 0.5, c'est la catastrophe : la loss part dans le décor, elle grimpe au lieu de descendre. Pourquoi ? Le pas est devenu si grand qu'il dépasse le fond de la vallée à chaque fois et rebondit d'un versant à l'autre, de plus en plus haut. C'est toute la difficulté du learning rate : trop petit, on apprend à la vitesse d'un escargot ; trop grand, on diverge. Le bon réglage vit dans une fenêtre étroite, et le trouver fait partie du métier.

Pour voir un peu plus large que le cours, les descriptions d'architecture disponibles en ligne :





Creusons

"Mais attends, du coup la backpropagation, c'est juste mathématiquement de la dérivée en chaîne ?"

Oui, exactement. L'idée a été formalisée par [Rumelhart, Hinton et Williams en 1986](#), dans un papier de quatre pages dans Nature. La beauté, c'est que le framework (TensorFlow, PyTorch) calcule tous ces gradients pour vous, automatiquement, peu importe la taille du réseau. Vous décrivez l'architecture, il s'occupe de la dérivation. Ce que vous venez de coder à la main sur un poids, le framework le fait sur des millions.

SGD ou Adam ? Adam converge plus vite en général, donc on commence avec lui. Mais sur certains problèmes, un SGD bien réglé généralise mieux. Ce n'est pas une règle absolue : c'est une raison de plus de mesurer et comparer, le réflexe Arène de toute la semaine. Vous pourrez justement les comparer cet après-midi.

Dernier point d'attention pour la suite : trop entraîner un réseau peut le faire **surapprendre** (overfitting), c'est-à-dire mémoriser les données d'entraînement par cœur au lieu de généraliser. C'est tout le sujet de l'évaluation, qu'on attaque cet après-midi.

On a tout disséqué à la main : forward, loss, gradient, backprop. Maintenant la bonne nouvelle, celle qui fait sourire après tout ce détail : en pratique, on ne code rien de tout ça. Un framework le fait pour nous en cinq lignes. Voyons le PMC pour de vrai.

6 - Le PMC en vrai : Keras, et quand l'utiliser

Tout ce qu'on vient de voir (poids, activations, loss, backprop, optimiseur) tient en quelques lignes de [Keras](#), l'interface haut niveau de TensorFlow. L'analogie : Keras est le volant, TensorFlow est le moteur. On conduit, on ne démonte pas le moteur.

L'essentiel

On construit un PMC pour de la classification binaire (tumeur bénigne ou maligne), sur un dataset jouet de scikit-learn. Voici ce que le code enchaîne, et chaque étape est un concept du matin qui revient :

- charger les 569 patients et leurs 30 mesures, séparer train/test,
- normaliser les données (réflexe de toute la semaine : un réseau déteste les échelles disparates),
- empiler le réseau avec `Sequential` : deux couches cachées ReLU, une sortie sigmoïde (1 neurone + sigmoïde = une probabilité, donc du binaire),
- compiler en choisissant l'optimiseur (`adam`), la loss (`binary_crossentropy`, la BCE de la section 5) et la métrique à suivre,
- entraîner avec `fit` sur 20 epochs : c'est lui qui exécute forward, loss, backprop et mise à jour, vingt fois,
- évaluer sur le test, des données que le réseau n'a jamais vues.

Lisons les commentaires, on reconnaîtra chaque brique :

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow import keras
from tensorflow.keras import layers

# 1. Données : 569 patients, 30 mesures, classe 0 (maligne) ou 1 (bénigne)
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Normalisation : indispensable pour un réseau (échelles homogènes)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Construire le réseau : 2 couches cachées ReLU, 1 sortie sigmoïde
# (sigmoïde + 1 neurone = on veut une probabilité, classification binaire)
model = keras.Sequential([
    layers.Input(shape=(30,)),          # 30 features en entrée
    layers.Dense(16, activation="relu"), # couche cachée 1
    layers.Dense(8, activation="relu"),  # couche cachée 2
    layers.Dense(1, activation="sigmoid") # sortie : probabilité
])

# 4. Compiler : on choisit l'optimiseur, la loss et la métrique à suivre
# binary_crossentropy = la BCE vue en section 5
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

# 5. Entraîner : 20 passages sur les données (epochs)
```



```

history = model.fit(X_train, y_train, epochs=20, batch_size=16, verbose=0)
# batch_size=16 : on montre 16 exemples au réseau avant chaque mise à jour des poids

# 6. Évaluer sur des données jamais vues
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy sur le test : {acc:.2%}")
print(f"Loss finale (train) : {history.history['loss'][-1]:.4f}")

```

Devrait afficher quelque chose comme (les chiffres varient un peu) :

```

Accuracy sur le test : 97.37%    <- accuracy : le % de bonnes réponses
Loss finale (train) : 0.0631

```

Reconnaissez le matin : `Dense` est une couche de neurones "fully connected", `relu` en cachées, `sigmoid` en sortie, `binary_crossentropy` la loss, `adam` l'optimiseur, `epochs` le nombre de tours de la boucle d'entraînement. `model.fit()` exécute pour vous forward, loss, backprop, mise à jour, vingt fois. Toute la section 5 dans une ligne.

C'est quoi `Sequential` ? La façon Keras de construire un réseau couche après couche, dans l'ordre, comme une pile. Parfait pour un PMC où l'info va tout droit.

Et `Dense` ? Une couche dense (fully connected) : chaque neurone est relié à tous ceux de la couche précédente. C'est exactement le forward pass `données * poids + biais` de la section 3.

On bouge un paramètre : relançons en changeant la première couche cachée de `Dense(16, ...)` à `Dense(64, ...)`, puis à `Dense(4, ...)`. On va remarquer que l'accuracy de test bouge à peine : sur un dataset aussi petit et facile, élargir la couche ne change presque rien, le réseau avait déjà largement de quoi faire. Puis remplaçons `activation="relu"` par `activation="sigmoid"` dans les couches cachées et regardons la loss finale. Là, elle apprend souvent moins bien, la loss reste plus haute : on vient de voir le vanishing gradient de la section 4 en action, ces sigmoïdes empilées qui aplatissent les gradients et freinent l'apprentissage des premières couches. C'est exactement pour ça que ReLU est le défaut en couches cachées.

Quand le PMC gagne, quand il est inutile. C'est LE point de jugement de la journée :

Situation	Le bon choix	Pourquoi
Petites données tabulaires (un Excel, < 10 000 lignes)	Random Forest / Gradient Boosting	Plus rapide, plus précis, explicable. Le PMC ici, c'est du gâchis.
Images, son, texte brut	Réseau de neurones	Le ML classique ne sait pas lire des pixels bruts.
Beaucoup de données + motifs très complexes	Réseau de neurones	Il a de quoi se nourrir et peut capter la complexité.
Besoin d'expliquer la décision (banque, santé, légal)	ML classique	Un PMC est une boîte noire, difficile à justifier devant un régulateur.

Sur le `breast_cancer` qu'on vient de faire (569 lignes, données tabulaires), un Random Forest atteint la même accuracy, s'entraîne en une fraction de seconde et vous dit quelles mesures comptent le plus. Le PMC est un marteau magnifique, mais tout n'est pas un clou.

Approfondissons

Un piège classique qu'on n'a pas le temps d'éviter ici mais qu'il faut connaître : on lit l'accuracy sur le **train** dans le `history`, et l'accuracy sur le **test** dans `evaluate`. Si la première est très haute (99%) et la seconde basse (80%), votre réseau a **surappris** : il a mémorisé l'entraînement sans généraliser. C'est exactement le danger que l'évaluation rigoureuse de cet après-midi va nous apprendre à détecter et à mesurer.

Et si vous voulez voir la boucle d'entraînement de l'intérieur en Keras, regardez `history.history['loss']` : c'est la liste de la loss à chaque epoch. Tracez-la (avec matplotlib) et vous verrez la courbe descendre, exactement comme dans le petit numpy de la section 5, mais sur un vrai réseau. C'est la même mécanique, à une autre échelle.

À votre tour

Ce matin, on a découvert le réseau de neurones et on l'a fait tourner. ~~Cet après-midi~~ Ce matin toujours, vous pilotez, et on change de casquette : on devient **arbitres**. Avoir des combattants dans l'Arène, c'est bien. Les juger correctement et déployer le vainqueur, c'est le vrai métier de data scientist.

Le projet

Vous prenez un dataset de classification, vous **évaluez** plusieurs modèles avec une rigueur de pro (séparation propre des données, validation croisée, choix de la métrique selon le coût métier), puis vous **déployez** le champion : vous le sauvegardez, vous le servez derrière une API, et vous montez une petite WebApp de prédiction. À la fin, votre modèle vit sur le web et n'importe qui peut lui envoyer un cas et recevoir une prédiction.

Le livrable : un notebook d'évaluation propre, un script de sauvegarde du modèle, une API minimale, une WebApp Streamlit ou Gradio, et un README qui explique vos choix. Le tout poussé sur GitHub.

Ici, vous avez les signatures, les sorties attendues et les étapes. À vous d'écrire le code. Bloquer un peu, c'est le but.

Phase 0 : Mise en route

L'intendance d'abord, une bonne fois. Le code de cet après-midi a besoin de quelques bibliothèques :

- Travaillez dans [Google Colab](#) (plus simple car tout est préinstallé) ou en local, au choix !
- Si en local, installez ce qu'il faut : `pip install scikit-learn tensorflow joblib flask streamlit gradio`.
- Reprenez votre repo GitHub de la semaine, créez un dossier `jour4/`. Premier commit : un README qui décrit le plan de l'après-midi.

Note dépôt : votre repo est votre copie d'examen continue. On note les commits, pas seulement le résultat. Committez à chaque phase finie, message clair, fichiers ajoutés un par un (pas de `git add .` à l'aveugle).

Jeu de données:

Vous le verrez, ce projet est sur les données cancer, mais libre à vous de faire cela sur un autre jeu de données ! (Plus fun que de rester plusieurs jours sur le même exemple). Si vous changez de jeu, **trois contraintes** simples à vérifier avant de commencer :

1. Classification binaire : il faut avoir une colonne cible à deux valeurs
2. Au moins 500 lignes : c'est la taille minimum du jeu
3. Mix de colonnes numériques et catégorielles (sinon les phases d'encodage sont triviales)

Si vous voulez changer mais que vous ne trouvez rien, vous pouvez aussi passer sur : Titanic, Heart Disease, Bank Marketing, tous les trois sur Kaggle.

Phase 1 : Séparer les données proprement, train / validation / test

On ne juge jamais un modèle sur les données qui ont servi à l'entraîner. Ce serait noter un élève sur les exercices qu'il avait en révision. Trois jeux, trois rôles :

- **Train** : le modèle apprend dessus.
- **Validation** : on règle les choix (quel algo, quels réglages) en regardant ce jeu, sans jamais l'entraîner dessus.
- **Test** : on n'y touche qu'une seule fois, tout à la fin, pour l'estimation honnête de la performance réelle.

Pourquoi un jeu de validation en plus du test ? Si on règle nos choix en regardant le test, on "triche" : on adapte le modèle au test, qui n'est plus une surprise. Le validation sert de terrain d'entraînement à nos décisions, le test reste vierge pour le verdict final.

```
from sklearn.model_selection import train_test_split

def split_train_val_test(X, y, test_size=0.2, val_size=0.2, random_state=42):
    """Découpe X, y en trois jeux : train, validation, test.
    Doit renvoyer 6 objets : X_train, X_val, X_test, y_train, y_val, y_test.
    Les proportions doivent rester respectées (utiliser stratify=y).
    """
    # TODO : premier split pour isoler le test (test_size)
    # TODO : deuxième split sur le reste pour isoler la validation
    # TODO : penser à recalculer la proportion val sur le reste
    # TODO : renvoyer les 6 objets
    # stratify=y : garder la même proportion de classes dans chaque jeu
    # qu'au départ, pour ne pas créer un test sans la classe rare
    pass
```

Devrait afficher, sur un dataset de 569 lignes :

```
Train : 364 | Validation : 91 | Test : 114
Répartition des classes conservée dans chaque jeu : oui
```

Checkpoint qualité, testez trois cas :

- cas normal : les trois tailles s'additionnent bien au total.
- cas limite : passez un `val_size=0` : votre fonction plante-t-elle proprement ou silencieusement ?
- cas adversarial : un dataset à 95% d'une seule classe. Vérifiez que `stratify` garde bien les 95/5 dans les trois jeux. Sans stratification, le test pourrait n'avoir aucune classe minoritaire : votre métrique serait un mensonge.

Phase 2 : Bootstrap et bagging, comprendre le rééchantillonnage

Une estimation sur un seul découpage, c'est fragile : changez la graine du hasard, le score bouge. Le [bootstrap](#) répond à ça : on tire des échantillons **avec remise** (un même point peut sortir plusieurs fois) pour fabriquer plein de jeux légèrement différents, et on regarde la dispersion des scores.

Avec remise ? À chaque tirage on remet le point dans le sac avant le suivant. Donc un échantillon bootstrap peut contenir deux fois la ligne 7 et zéro fois la ligne 12. C'est ce qui crée la variété.

Le **bagging** (Bootstrap AGGREGatING) applique cette idée pour fabriquer un modèle : entraîner plein de modèles sur des échantillons bootstrap différents, puis faire voter. C'est exactement ce qu'un Random Forest fait sous le capot (vu en J3). Ici, on le rend visible.

```
import numpy as np

def bootstrap_scores(modele, X, y, n_iterations=30, random_state=42):
    """Évalue la stabilité d'un modèle par bootstrap.
    Pour chaque itération : tirer un échantillon AVEC REMISE de même taille
    que X, entraîner, évaluer sur les points NON tirés (out-of-bag).
    Doit renvoyer la liste des scores et afficher moyenne et écart-type.
    """
    # TODO : rng = np.random.default_rng(random_state)
    # TODO : pour chaque itération, tirer des indices avec remise (rng.choice avec
    replace=True)
    # TODO : identifier les indices out-of-bag (OOB) pour évaluer
    #         OOB = les points jamais tirés dans cet échantillon bootstrap,
    #         ils n'ont pas servi à l'entraînement : jeu de test gratuit
    # TODO : entraîner sur l'échantillon, scorer sur l'out-of-bag
    # TODO : renvoyer la liste, afficher moyenne ± écart-type
    pass
```

Devrait afficher quelque chose comme :

```
Score moyen sur 30 bootstraps : 0.952 (± 0.018)
```

Ce qui doit casser, et que vous devez gérer : que se passe-t-il si une itération produit un échantillon out-of-bag vide (rare mais possible) ? Et si vous oubliez `replace=True`, votre "bootstrap" devient un simple mélange : comparez l'écart-type avec et sans remise, vous verrez la différence. Testez aussi avec `n_iterations=1` : un seul score, aucun écart-type, est-ce que votre affichage reste honnête ?

Phase 3 : La validation croisée k-fold

Le bootstrap, c'est une façon de mesurer la stabilité. La **validation croisée k-fold** en est une autre, la plus standard. On découpe les données en k paquets (folds). On entraîne k fois, à chaque fois sur k-1 paquets et on teste sur celui qu'on a mis de côté. Chaque point sert une fois au test. On obtient k scores, donc une moyenne ET une dispersion fiables.

scikit-learn fait ça en une ligne avec `cross_val_score` :

```

from sklearn.model_selection import cross_val_score

def evaluer_en_cross_val(modele, X, y, k=5):
    """Lance une validation croisée k-fold et résume les résultats.
    Doit afficher les k scores, leur moyenne et leur écart-type.
    Un grand écart-type entre folds = modèle instable, à signaler.
    """
    # TODO : scores = cross_val_score(modele, X, y, cv=k, scoring="accuracy")
    # TODO : afficher les k scores
    # TODO : afficher moyenne et écart-type, et commenter la stabilité
    pass

```

Devrait afficher :

```

Scores par fold : [0.956 0.939 0.965 0.947 0.956]
Moyenne : 0.953 | Écart-type : 0.009 -> modèle stable

```

Trois situations à vérifier :

- cas normal : 5 folds, une moyenne et un petit écart-type.
- cas limite : `k` égal au nombre de lignes (validation croisée "leave-one-out"). Ça tourne, mais c'est lent : pourquoi ? Mesurez le temps.
- cas adversarial : sur le dataset déséquilibré (95/5), comparez `cross_val_score` standard avec `StratifiedKFold`. Sans stratification, un fold peut ne contenir aucun cas de la classe rare et fausser la moyenne.

Phase 4 : Choisir la bonne métrique selon le coût métier

Pour creuser : [StatQuest - ROC and AUC](#) (~16 min) : l'explication la plus claire qui existe sur ce que la courbe ROC mesure vraiment, et pourquoi l'AUC seule peut aussi mentir.

Voici la phase qui sépare un junior d'un pro. *L'accuracy (le pourcentage de bonnes réponses) suffit-elle à juger un modèle ?* Non. Et l'exemple qui le prouve est promis depuis J1.

L'histoire des 99% qui sont une catastrophe. Vous détectez une fraude bancaire. Sur 10 000 transactions, 100 sont frauduleuses (1%). Un modèle paresseux qui répond toujours "pas de fraude" a une accuracy de... 99%. Magnifique sur le papier. Sauf qu'il laisse passer 100% des fraudes. L'accuracy est un mensonge sur données déséquilibrées.

Ce qui compte, c'est le **coût métier de chaque type d'erreur** (les métriques elles-mêmes, matrice de confusion, [precision, recall, F1, ROC/AUC](#), lift, ont été vues en J3, on les réutilise) :

- Rater une fraude (faux négatif) coûte très cher : on veut un **recall élevé**.
- Sur un dépistage médical, rater un malade (faux négatif) est dramatique : recall avant tout.
- Sur un filtre anti-spam, mettre un vrai mail important en spam (faux positif) est pénible : on veut une **precision élevée**.

```

from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score

def rapport_metier(y_true, y_pred, cout_fn=10, cout_fp=1):
    """Affiche la matrice de confusion et les métriques, puis calcule un
    COÛT MÉTIER total : (nb faux négatifs * cout_fn) + (nb faux positifs * cout_fp).
    Doit afficher precision, recall, F1, et le coût total.
    Permet de comparer deux modèles sur l'argent perdu, pas sur l'accuracy.
    """
    # TODO : matrice = confusion_matrix(y_true, y_pred) -> en extraire VN, FP, FN, VP
    # TODO : afficher precision, recall, f1
    # TODO : cout_total = FN * cout_fn + FP * cout_fp
    # TODO : afficher le coût métier total
    pass

```

Devrait afficher, en comparant deux modèles :

```

Modèle A : precision=0.91 recall=0.62 F1=0.74 | coût métier = 395
Modèle B : precision=0.78 recall=0.94 F1=0.85 | coût métier = 142
-> Le modèle B est moins précis mais coûte 2.7x moins cher en fraude ratée.

```

Le scénario qui pique, en trois temps :

- cas normal : deux modèles raisonnables sur le dataset de fraude. Affichez pour chacun precision, recall, F1 ET coût métier. Vérifiez que celui qui a la meilleure accuracy n'est pas forcément celui qui a le plus petit coût : c'est tout l'intérêt de la phase.
- cas limite : le modèle "toujours pas de fraude" sur le dataset déséquilibré. Affichez son accuracy (élevée) ET son recall (zéro) côte à côte. Vérifiez aussi que votre fonction ne plante pas quand une classe n'est jamais prédite (un `precision_score` peut renvoyer une division par zéro : gérez le `zero_division`).
- cas adversarial : un collègue vous présente fièrement "99% d'accuracy" et veut déployer. Votre job d'arbitre : exigez la matrice de confusion, calculez le coût métier réel, et montrez chiffres à l'appui qu'un modèle moins "précis" mais à fort recall coûte beaucoup moins cher en fraude ratée. C'est la démonstration qui doit tenir devant un décideur, pas juste un tableau de métriques.

Phase 5 : Sérialiser le modèle et le servir derrière une API

Un modèle entraîné qui meurt à la fin du notebook ne sert à rien. Il faut le **sérialiser** (le sauvegarder sur disque) puis le **recharger** ailleurs pour s'en servir.

Sérialiser ? Transformer un objet en mémoire (votre modèle entraîné) en un fichier qu'on peut stocker et recharger plus tard. Pour scikit-learn on utilise `joblib` (fichier `.joblib`). Pour Keras, la méthode `model.save("modele.keras")`.

Petit rappel API, parce qu'on va en monter une :

Une API, c'est quoi déjà ? Une porte d'entrée pour parler à un programme à distance. Le client envoie une **requête** (une URL appelée **endpoint**, avec des données au format **JSON**), le serveur répond avec un résultat (du JSON aussi). "Voici les mesures du patient" en entrée, "probabilité de tumeur maligne" en sortie.

On sauvegarde le modèle, puis on monte un endpoint minimal avec [Flask](#) (un micro-framework web Python). Le client envoie des features, l'API renvoie une prédiction.

```

import joblib
from flask import Flask, request, jsonify

def sauvegarder_modele(modele, scaler, chemin="modele.joblib"):
    """Sauvegarde le modèle ET le scaler ensemble (sinon les prédictions
    en prod seront fausses : il faut normaliser pareil qu'à l'entraînement).
    """
    # TODO : joblib.dump({"modele": modele, "scaler": scaler}, chemin)
    pass

app = Flask(__name__)
# TODO : au démarrage, recharger le dict avec joblib.load(...)

@app.route("/predict", methods=["POST"])
def predict():
    """Reçoit un JSON {"features": [...]}, renvoie {"prediction": ..., "proba": ...}.
    Doit : lire le JSON, normaliser avec le scaler, prédire, renvoyer du JSON.
    """
    # TODO : data = request.get_json()
    # TODO : valider que "features" existe et a la bonne longueur
    # TODO : normaliser, prédire, renvoyer jsonify({...})
    pass

```

Devrait répondre, à un `curl -X POST .../predict -d '{"features": [...]}'` :

```
{"prediction": 1, "proba": 0.97, "label": "benigne"}
```

Checkpoint, trois requêtes à tester :

- cas normal : un vrai vecteur de features, on reçoit une prédiction cohérente.
- cas limite : un JSON sans clé `features`, ou avec un mauvais nombre de valeurs. L'API doit renvoyer une erreur claire (code 400), pas planter.
- cas adversarial : quelqu'un envoie du texte au lieu de nombres, ou un tableau vide. Votre validation tient-elle ? Une API en prod reçoit toujours des données pourries un jour.

Phase 6 : Déployer une WebApp de prédiction

L'API, c'est pour les machines. Pour un humain, on veut une interface. [Streamlit](#) (ou [Gradio](#)) permet de monter une WebApp de prédiction en quelques lignes, sans HTML.

Streamlit / Gradio ? Deux outils Python qui transforment un script en page web interactive : des champs de saisie, un bouton, un résultat affiché. Zéro HTML/JS à écrire. Parfait pour démontrer un modèle.

```
import streamlit as st
import joblib

def lancer_webapp():
    """Charge le modèle, affiche des champs de saisie pour les features,
    un bouton 'Prédire', et le résultat avec sa probabilité.
    Lancement : streamlit run app.py
    """
    # TODO : charger le dict modèle+scaler avec joblib.load
    # TODO : st.title(...) et des st.number_input(...) pour les features clés
    # TODO : au clic sur st.button("Prédire"), normaliser, prédire, st.write le résultat
    # TODO : afficher la probabilité (st.metric ou st.progress)
    pass
```

Au lancement, devrait ouvrir une page avec les champs et, après un clic :

```
Prédiction : tumeur bénigne (probabilité 97%)
```

Le projet final qui doit tenir debout :

- cas normal : on saisit des valeurs plausibles, on obtient une prédiction lisible.
- cas limite : on laisse un champ vide ou on saisit une valeur négative impossible. L'app prévient, elle ne crashe pas.
- cas adversarial : on saisit des valeurs aberrantes (une surface de tumeur de 99999). Le modèle prédira quand même quelque chose : affichez un avertissement quand une entrée est hors de la plage vue à l'entraînement. Un modèle ne sait pas dire "je ne sais pas", c'est à vous de le garder.

 **Bonus perso** : votre modèle peut vivre sur le web ce soir. Streamlit propose un déploiement gratuit (Streamlit Community Cloud) connecté à votre repo GitHub. Mini-défi hors-cours : déployez votre WebApp et envoyez le lien à un proche. Pas noté, juste la satisfaction de voir votre champion en ligne.

Phase 7 : L'arbitrage final (phase ouverte)

Cette phase n'a pas de ligne d'arrivée. Tant qu'il vous reste du temps, vous creusez l'évaluation et la comparaison, en vrai arbitre de l'Arène.

- Faites s'affronter votre **meilleur algo classique de J3** (Random Forest, par exemple) contre le **PMC Keras du matin** sur le même dataset, en validation croisée. Qui gagne en accuracy ? Et en coût métier ? Et en temps d'entraînement ?
- Comparez **SGD vs Adam** sur le PMC : tracez les deux courbes de loss, lequel converge le plus vite ?
- Sur le déploiement : mesurez la **latence** de votre API (temps entre requête et réponse) et discutez de la **fraîcheur du modèle** (un modèle entraîné il y a un an sur des données qui ont changé prédit de plus en plus mal : c'est le drift) et du **monitoring** de base (logger chaque prédiction pour détecter quand le modèle dérape).

Le leaderboard de l'arbitre (agrégation des métriques). Rassemblez tout dans un seul tableau récapitulatif, qui agrège vos mesures à travers les phases. C'est le livrable qui prouve que vous savez juger, pas juste entraîner :

Modèle	Accuracy (cross-val)	Recall	Coût métier	Temps d'entraînement	Latence prédiction
Random Forest	...	...	...	...	...
Gradient Boosting	...	...	...	...	...
PMC (Keras)	...	...	...	...	...

Remplissez-le pour vos modèles. Et tranchez : quel champion déployez-vous, et pourquoi ? Sur ce dataset tabulaire, ne soyez pas surpris si un algo classique bat le PMC tout en étant dix fois plus rapide. C'est exactement la leçon de la journée.

Peer review (relecture par deux)

Avant de committer la version finale, échangez avec un binôme. Vous vous relisez avec cette grille (ce n'est pas une note, c'est une discussion) :

- La séparation train / validation / test est-elle propre, sans fuite (aucune donnée de test vue avant le verdict) ?
- La métrique choisie correspond-elle au coût métier du problème, ou est-ce juste l'accuracy par réflexe ?
- L'API gère-t-elle une requête malformée sans planter ?
- Le choix du champion est-il justifié par les chiffres, ou par "j'aime bien le réseau de neurones" ?
- Y a-t-il des choix différents des vôtres qui sont tout aussi valables ? Lesquels, et pourquoi ?

Le but : voir qu'en ML il n'y a pas une seule bonne réponse. Le découpage, la métrique, l'algo retenu, tout se discute et se justifie.

Révision et QCM

On boucle la partie théorique de la semaine. Le QCM individuel en ligne valide vos acquis. Son **périmètre** :

- le ML classique (J1 à J3) : familles d'apprentissage, CRISP-DM, preprocessing, les algos (régressions, arbres, Random Forest, boosting, SVM, KMeans), les métriques (matrice de confusion, precision/recall, ROC/AUC).
- l'**introduction** au Deep Learning de ce matin : architecture du PMC, fonctions d'activation et leur usage, intuition de la loss, backpropagation et descente de gradient, quand un PMC bat un algo classique et quand il ne sert à rien.
- l'évaluation et la production de cet après-midi : train/val/test, bootstrap/bagging, cross-validation, choix de la métrique, sérialisation et déploiement.

Ce qui n'est **pas au programme** (c'est de la 5e année) : les réseaux convolutifs (CNN), les réseaux récurrents (RNN/LSTM), le transfer learning, les architectures profondes spécialisées. Si une question vous semble porter là-dessus, c'est hors programme pour nous.

Le QCM se fait en ligne. Révisez le récapitulatif ci-dessous avant de le lancer.

Récapitulatif Jour 4

Ce qu'on a appris aujourd'hui

1. **D'où vient le Deep Learning** : une vieille idée (1943, 1958, 1986) débloquée vers 2012 par trois ingrédients : données massives, GPU, frameworks.
2. **L'architecture du PMC** : entrée, couches cachées, sortie. Un neurone = somme pondérée + biais + activation. On configure la sortie selon la tâche (régression, binaire, multiclass).
3. **Les fonctions d'activation** : ReLU en couches cachées, sigmoïde (binaire) ou softmax (multiclass) en sortie, rien (linéaire) pour la régression. Sans elles, le réseau reste linéaire.
4. **Comment un réseau apprend** : loss (mesurer l'erreur), gradient (direction de correction), backpropagation (distribuer le blâme), optimiseur (faire le pas). SGD et Adam. La loss descend, epoch après epoch.
5. **Le PMC en Keras** : `Sequential`, `Dense`, `compile`, `fit`, en cinq lignes. Et surtout : quand l'utiliser (images, son, texte, gros volumes) et quand un algo classique gagne (petites données tabulaires, besoin d'explicabilité).
6. **L'évaluation rigoureuse** : train/validation/test, bootstrap et bagging, validation croisée k-fold, et le choix de la métrique selon le coût métier (les 99% d'accuracy qui sont une catastrophe).
7. **La mise en production** : sérialiser un modèle, le servir derrière une API, déployer une WebApp Streamlit/Gradio, et penser latence, fraîcheur, monitoring.

Points d'attention pour demain

- Si votre WebApp et votre notebook d'évaluation ne sont pas committés sur GitHub, faites-le : c'est la trace de votre travail, et c'est noté.
- Demain, **J5, c'est le grand jour** : projet en équipe, de bout en bout. Chaque groupe choisit son problème, monte sa propre Arène, désigne et déploie son champion, et présente en soutenance.
- Gardez en tête la leçon centrale de la semaine : le meilleur modèle n'est pas le plus sophistiqué, c'est celui qui répond au besoin métier, qu'on sait justifier, et qu'on a su évaluer proprement. Un projet 100% ML classique, sans réseau de neurones, est parfaitement valide.
- Relisez ce récapitulatif et celui des jours précédents avant le QCM.